Bonus Assignment | Section - A | OS Monsoon'2025

SimpleOS

A lightweight OS with an MLFQ scheduler and custom shell commands

Important Instructions

Due by **11:59PM** by **End of Semester**

(Total **4%** Weightage)

Instructor: Vivek Kumar

No extensions will be provided. Any submission after the deadline will not be evaluated. If you see an ambiguity or inconsistency in a question, please seek clarification from the teaching staff.

Plagiarism: This is a pair programming-based assignment that you must do with the group member that you have already chosen. No change to the group is allowed. You are not allowed to discuss the approach/solution outside your group. You should never misrepresent some other group's work as your own. In case any plagiarism case is detected, it will be dealt as per the new plagiarism policy of IIITD **and will be applied to each member in the group**. Even if you are a single member group, there will not be any relaxations in marking scheme/deadlines, and the same rubric will be followed for each group.

Open-sourcing of this assignment solution is not allowed, even after the course gets over.

General Instructions

1. Hardware requirements:
    1. You will require a machine having any Operating System that supports Unix APIs. You should not use MacOS for OS assignments. You can either have a dual boot system having any Linux OS (e.g., Ubuntu), or install a WSL.
2. Software requirements are:
    1. C compiler and GNU make.
    2. You **must** do version controlling of all your code using github. You should only use a **PRIVATE** repository. If you are found to be using a **PUBLIC** access repository, then it will be considered plagiarism. NOTE that TAs will check your github repository during the demo.

Overview

This assignment focuses on exploring both **user space** and **kernel space** programming within the **EGOS-2000 operating system.** It is divided into two parts, the implementation of **custom shell commands** and the development of a **Multi-Level Feedback Queue (MLFQ) scheduler**.

GitHub Repo → https://github.com/yhzhang0128/egos-2000/

The first part involves building simplified shell utilities (grep and wcl), emphasizing the use of system calls and low-level file operations to understand how user applications interact with the underlying OS. The second part focuses on implementing a preemptive MLFQ scheduler within the kernel to manage process execution based on priority and runtime behavior.

Take a good look at the Tutorial - 7 and the EGOS book (https://egos.fun/book/overview.html) for a better understanding of the OS.

Compilation and Setup

Follow → https://github.com/yhzhang0128/egos-2000/blob/main/USAGES.md

Step 1 → Setup the compiler and compile egos-2000

Setup your working directory and name it as $EGOS.

export EGOS=/home/yunhao/egos

cd $EGOS

git clone <https://github.com/yhzhang0128/egos-2000.git>

Download the pre-built binaries of the RISC-V GNU compiler toolchain from xPack to $EGOS and compile egos-2000.

cd $EGOS

tar -zxvf xpack-riscv-none-elf-gcc-14.2.0-3-{linux,darwin}-{x64,arm64}.tar.gz

export PATH=$PATH:$EGOS/xpack-riscv-none-elf-gcc-14.2.0-3/bin

cd $EGOS/egos-2000

make

....

Step 2 → Run egos-2000 on the QEMU emulator

Download the pre-built binaries of QEMU from xPack.

cd $EGOS

tar -zxvf xpack-qemu-riscv-8.2.2-1-xxxxxx.tar.gz

export PATH=$PATH:$EGOS/xpack-qemu-riscv-8.2.2-1-xxxxxx/bin

cd $EGOS/egos-2000

make qemu

-------- Simulate on QEMU-RISCV --------

qemu-system-riscv32 -nographic -readconfig tools/qemu/config.toml

[CRITICAL] --- Booting on QEMU with core #4 ---

[INFO] Set the CS pin to HIGH and toggle clock

[INFO] Set the CS pin to LOW and send cmd0 to SD card

[INFO] Check SD card type and voltage with cmd8

[INFO] SD card replies cmd8 with status 1

[SUCCESS] Finished initializing the tty and disk devices

[CRITICAL] Choose a memory translation mechanism:

Enter 0: page tables

Enter 1: software TLB

You can choose 0 (page tables) for the memory implementation.

Custom Shell Commands **(Weightage: 1 marks)**

Objective

You will implement two minimal shell utilities in the EGOS-2000 environment →

- grep → search for a pattern in a file and print matching lines.
- wcl → count the total number of lines in a file (equivalent to wc -l present in the bash shell).

Environment & Constraints

- Allowed headers →
- #include "app.h" // refer egos-2000/apps/user/app.h
- #include <string.h>
- You may use the following system call wrappers provided in **library/syscall/servers.c →**
    - dir_lookup(int dir_ino, char *name) → retrieves the inode of a file in the working directory.
    - file_read(int file_ino, uint offset, char *block) → reads one block (of size BLOCK_SIZE) from a file starting at the given offset.
- Refer to **egos-2000/apps/user/cat.c** to understand how these system calls are made from scratch.
    - **Important →** Notice that cat.c reads only the *first block* of a file (using file_read(file_ino, 0, buf)).
    - In this assignment, **you must not stop at the first block,** instead, **read the entire file**.
- Do **not** use any standard C file APIs (fopen, fread, fgets, etc.) or Linux syscalls. All I/O must go through the EGOS system calls.

Specification→ grep

Usage

grep [PATTERN] [FILE]

Behavior

1. Takes exactly two arguments →
    1. PATTERN → the substring to search for.
    2. FILE → the file to search in. (you may assume files are textual (no binary content) and the file may or may not be empty)
2. Use dir_lookup() to obtain the inode number of FILE.
3. Read the **entire file** using file_read().
4. Process it to search for lines containing PATTERN.
5. If PATTERN is found in that line, print that line followed by a newline.
6. If the pattern is not found, print nothing (but exit successfully).

Specification: wcl

Usage

wcl [FILE1] [FILE2] ....

Behavior

1. Can takes many files as arguments → the name of the each file.
2. For each file →
    1. Use dir_lookup() to get the file inode.
    2. Read **the entire file** using file_read().
    3. Count the number of lines in the content.
3. Print the total line count of all the files, followed by a newline.

MLFQ Scheduler **(Weightage: 3 marks)**

Objective

1. All changes to the code are to be made in the **Grass** layer itself and not in any other component.
2. The OS raises a **timer interrupt** after a set period of time after which execution of the currently running process is halted and a new process is selected for execution.
3. The students are not required to make any updations to the **interrupt** mechanism as it comes fully functional out of the box with EGOS.
4. Only complete the code in empty places where **(Preemptive Scheduler)** is mentioned and not anywhere else.

General outline and steps

Update the process struct in process.h

- The **process** struct contains information such as process id, status and registers used etc. for each process on EGOS.
- You will have to update the **process** struct so that it also stores the following →
    - Lifecycle data for the process →
        - **Turnaround time →** the time between process creation and termination
        - **Response time →** the time between process creation and the first time scheduled
        - **CPU time →** the accumulated time that the process is actually running on the CPU
        - **Number of timer interrupts encountered →** how many times the CPU control flow transfers from the process code to trap_entry due to a timer interrupt
    - Information necessary for MLFQ scheduling →
        - Queue level
        - Remaining runtime on level
        - Any other details if required

Complete functions in process.c

- Two functions ; **mlfq_update_level()** and **mlfq_reset_level()** will play a central role in the implementation.
- You will have to implement the above functions so that the given rules are followed correctly (Rule Numbers are referred from the lecture slides in brackets) →
    1. There are 5 levels, level #0 to #4. All processes start at level #0 when created. (Rule 3)
    2. Processes at level #i can run on the CPU for (i+1)*100 milliseconds. For example, if a process at level #0 has run on the CPU for more than 100ms, it will move to level #1. After running for another 200ms, it will move to level #2. Since there are 5 levels, all processes at level #4 will remain at this level without moving further. (Rule 4)
    3. The scheduler in proc_yield should always choose a runnable process with the **lowest level number** (i.e., highest priority) as the next process. (Rule 1)
    4. Every 10 seconds or so, the scheduler in proc_yield should move all processes to level #0. Also, if there is keyboard input for the shell, move GPID_SHELL to level #0. (Rule 5)
- **mlfq_update_level()** should update the level of a process in accordance with Rule 4 and **mlfq_reset_level()** should help enforce Rule 5.

- There are two more functions ; **proc_alloc()** and **proc_free()** which initialize the lifecycle and MLFQ parameters and successfully terminate processes respectively.
- You should update **proc_free()** such that for every terminated process all the lifecycle statistics that were collected should be printed out.

Update and complete missing code segments in kernel.c

- **intr_entry()** defines code that is executed after an interrupt, complete this so that the lifecycle statistics for a process are updated initially.
- **proc_yield()** is an important function that selects and starts execution of a new process according to whatever your scheduling algorithm is.
- Update to **for()** loop in **proc_yield()** such that a runnable process with the **lowest level number** (i.e., highest priority) is chosen as the next process.
- Call **mlfq_update_level()** and **mlfq_reset_level()** in **proc_yield() .** You should decide where and when to call these two functions so that your implementation is correct.

Refer to (https://egos.fun/book/p2_mlfq.html) **Process lifecycle and priority** onwards for specific implementation details and a more in-depth guide.

You need to edit/add new files at appropriate locations to the cloned repo and submit the whole repo as a zip file.

Requirements

1. You should strictly follow the instructions provided above.
2. Proper error checking must be done at all places. It is up to you to decide what are those necessary checks.
3. Proper documentation should be done in your coding.
4. Your assignment submission should consist of two parts:
    1. A zip file containing your source files as mentioned above. Name the zip file as “group-ID.zip”, where “ID” is your group ID specified in the spreadsheet shared by the TF.
    2. A design document inside the above “zip” file detailing the contribution of each member in the group, detailing your SimpleOS implementation, and the link to your private github repository where your assignment is saved.
5. There should be ONLY ONE submission per group.
6. In case your group member is not responding to your messages or is not contributing to the assignment then please get in touch with the teaching staff immediately.
